

项目中导航栏的实现与第三方库使用

一、导航栏的核心实现

1. 自定义标题栏组件

项目采用 **自定义标题栏组件** 实现导航栏，核心文件为 `CustomTitleBar.java`，继承自 `RelativeLayout`，加载布局文件 `layout_titlebar.xml`。

1.1 组件结构与功能

Code block

```
1  public class CustomTitleBar extends RelativeLayout {
2      private ImageView ivBack, iv_nameicon;
3      private TextView tvTitle, tvMore, tvDemo, tvLogin;
4      public ImageView ivMore;
5      public ConstraintLayout cl_title;
6      public ImageView ivLogo;
7      public TextView tv_right, iv_bottomline;
8
9      // 构造方法与初始化
10     public CustomTitleBar(Context context, AttributeSet attrs) {
11         super(context, attrs);
12         this.context = context;
13         initView(context, attrs);
14     }
15
16     // 初始化视图
17     private void initView(final Context context, AttributeSet attributeSet) {
18         inflate =
19         LayoutInflater.from(context).inflate(R.layout.layout_titlebar, this);
20         // 查找并初始化所有子视图
21         // ...
22         init(context, attributeSet);
23     }
```

1.2 自定义属性与主题集成

`CustomTitleBar` 支持 XML 属性配置，与项目主题系统深度集成：

Code block

```

1  // 从属性集中获取配置
2  TypedArray typedArray = context.obtainStyledAttributes(attributeSet,
    R.styleable.CustomTitleBar);
3  String title = typedArray.getString(R.styleable.CustomTitleBar_title);
4  int leftIcon = typedArray.getResourceId(R.styleable.CustomTitleBar_left_icon,
    R.mipmap.finish);
5  int rightIcon =
    typedArray.getResourceId(R.styleable.CustomTitleBar_right_icon,
    R.mipmap.finish);
6  // ...
7
8  // 主题颜色应用
9  if (UGTheme.INSTANCE.getRnTheme() != null) {
10
    ivBack.setColorFilter(ColorUtil.INSTANCE.rnColor2AndroidColor(UGTheme.INSTANCE.
        getRnTheme().getNavBarTitleColor()));
11
    cl_title.setBackgroundColor(ColorUtil.INSTANCE.rnColor2AndroidColor(UGTheme.INS
        TANCE.getRnTheme().getNavSignBarBgColor()));
12 }

```

1.3 灵活的交互接口

提供丰富的点击事件接口，支持定制化交互：

Code block

```

1  // 左边图片点击事件
2  public void setLeftIconOnClickListener(OnClickListener l) {
    ivBack.setOnClickListener(l); }
3  // 右边图片点击事件
4  public void setRightIconOnClickListener(OnClickListener l) {
    ivMore.setOnClickListener(l); }
5  // 右边文字点击事件
6  public void setRightTextOnClickListener(OnClickListener l) {
    tvMore.setOnClickListener(l); }
7  // 其他交互方法...

```

2. 标题栏布局文件

布局文件 `layout_titlebar.xml` 采用 `ConstraintLayout` 实现，结构灵活：

Code block

```

1  <androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"

```

```

2      xmlns:app="http://schemas.android.com/apk/res-auto"
3      android:id="@+id/cl_title"
4      android:layout_width="match_parent"
5      android:layout_height="@dimen/dp_50"
6      android:background="@color/colorPrimary">
7
8      <!-- 返回按钮 -->
9      <ImageView android:id="@+id/iv_back" />
10
11     <!-- Logo -->
12     <ImageView android:id="@+id/iv_logo" />
13
14     <!-- 左侧文本 -->
15     <com.cryptoapp.view.UGTextView android:id="@+id/left_tex" />
16
17     <!-- 标题 -->
18     <com.cryptoapp.view.UGTextView android:id="@+id/tv_title" />
19
20     <!-- 右侧功能按钮 -->
21     <com.cryptoapp.view.UGTextView android:id="@+id/tv_demo" />
22     <com.cryptoapp.view.UGTextView android:id="@+id/tv_login" />
23     <com.cryptoapp.view.UGTextView android:id="@+id/tv_more" />
24     <com.cryptoapp.view.UGTextView android:id="@+id/tv_right" />
25
26     <!-- 更多图标 -->
27     <ImageView android:id="@+id/iv_more1" />
28     <ImageView android:id="@+id/iv_more" />
29
30     <!-- 底部分割线 -->
31     <TextView android:id="@+id/tv_bottomline" />
32 </androidx.constraintlayout.widget.ConstraintLayout>

```

2.1 布局特点

- 模块化设计：包含返回按钮、Logo、标题、右侧功能按钮等独立模块
- 灵活的显示控制：所有元素默认 `visibility="gone"`，可动态显示
- 响应式布局：使用 `maxWidth` 和 `ellipsize` 处理长文本
- 主题一致性：统一使用颜色和尺寸资源

3. 主题配置系统

项目通过 `ReactThemeBean.kt` 实现主题配置，包含导航栏相关属性：

Code block

```

1  data class ReactThemeBean(
2      // 导航栏背景颜色
3      @SerializedName("navBarBgColor")
4      var navBarBgColor: List<String>?,
5      // 导航栏标题颜色
6      @SerializedName("navBarTitleColor")
7      var navBarTitleColor: String?,
8      // 导航栏选中标题颜色
9      @SerializedName("navBarSelectTitleColor")
10     var navBarSelectTitleColor: String?,
11     // 导航栏未选中标题颜色
12     @SerializedName("navNoSelectTitleColor")
13     var navNoSelectTitleColor: String?,
14     // 导航栏签名栏背景颜色
15     @SerializedName("navSignBarBgColor")
16     var navSignBarBgColor: String?,
17     // 底部标签栏相关
18     @SerializedName("tabBarBgColor")
19     var tabBarBgColor: String?,
20     @SerializedName("tabSelectedColor")
21     var tabSelectedColor: String?,
22     @SerializedName("tabNoSelectColor")
23     var tabNoSelectColor: String?,
24     // ...其他主题属性
25 )

```

二、使用的第三方库

1. ImmersionBar - 沉浸式状态栏与导航栏

核心依赖

Code block

```

1  // 基础依赖包
2  implementation 'com.gyf.immersionbar:immersionbar:3.0.0'
3  // fragment快速实现（可选）
4  implementation 'com.gyf.immersionbar:immersionbar-components:3.0.0'
5  // kotlin扩展（可选）
6  implementation 'com.gyf.immersionbar:immersionbar-ktx:3.0.0'

```

使用方式

在 `BaseActivity.kt` 中初始化：

```
1 fun ImmersionBar() {  
2     ImmersionBar.with(this)  
3         .statusBarColor(R.color.colorPrimary) // 状态栏颜色  
4         .init() // 应用配置  
5 }
```

主要功能

- 实现状态栏和导航栏沉浸式效果
- 支持动态修改状态栏/导航栏颜色
- 丰富配置选项（透明度、字体颜色、布局适配等）
- 支持键盘适配和刘海屏处理

2. ConstraintLayout - 灵活的布局系统

核心优势

- 减少布局嵌套层级
- 支持复杂视图关系和定位
- 提供响应式设计能力
- 支持百分比布局和链式约束

3. 自定义 UGTextView

对原生 `TextView` 的扩展，提供额外功能和主题支持，适配项目统一风格。

三、导航栏的使用方式

1. 在布局文件中使用

Code block

```
1 <com.cryptoapp.view.CustomTitleBar  
2     android:id="@+id/titleBar"  
3     android:layout_width="match_parent"  
4     android:layout_height="wrap_content"  
5     app:title="页面标题"  
6     app:left_icon="@mipmap/back_icon"  
7     app:right_icon="@mipmap/more_icon"  
8     app:titlebar_type="11" />
```

2. 在代码中控制

```
1 CustomTitleBar titleBar = findViewById(R.id.titleBar);
2 titleBar.setTitle("新页面标题");
3 titleBar.setLeftIconOnClickListener(v -> finish());
4 titleBar.setRightIconOnClickListener(v -> showMenu());
```

3. 动态主题切换

通过 `UGTheme` 和 `ReactThemeBean` 实现主题动态更新：

Code block

```
1 // 当主题更新时，标题栏自动应用新颜色
2 if (UGTheme.INSTANCE.getRnTheme() != null) {
3
4     ivBack.setColorFilter(ColorUtil.INSTANCE.rnColor2AndroidColor(UGTheme.INSTANCE.
5         getRnTheme().getNavBarTitleColor()));
6
7     cl_title.setBackgroundColor(ColorUtil.INSTANCE.rnColor2AndroidColor(UGTheme.INS
8         TANCE.getRnTheme().getNavSignBarBgColor()));
9 }
```

四、设计特点与优势

1. 高度可定制化

- 支持 XML 属性和代码两种配置方式
- 丰富的显示控制和交互接口
- 可自定义每个元素的可见性和样式

2. 主题系统集成

- 与项目主题系统深度融合
- 支持动态主题切换
- 保证应用风格一致性

3. 组件化设计

- 组件化思想，提高代码复用性
- 封装通用功能，减少重复开发
- 清晰 API 接口，便于使用和扩展

4. 沉浸式体验

- 结合 ImmersionBar 实现沉浸式效果
- 提升视觉体验和屏幕利用率
- 适配各类设备和系统版本

五、优化建议

1. 性能优化

- 使用 `ViewStub` 延迟加载不常用视图元素
- 优化主题颜色应用逻辑，避免频繁 UI 更新

2. 代码结构优化

- 将主题相关逻辑抽取到单独类中，提升可维护性
- 考虑用 Kotlin 重写 `CustomTitleBar`，简化代码

3. 扩展性增强

- 增加更多自定义属性，支持更灵活配置
- 支持导航栏动画效果和过渡效果

4. 测试与稳定性

- 增加单元测试，确保组件稳定性
- 测试多设备、多系统版本兼容性

总结

项目导航栏采用 **自定义组件 + 主题系统 + 第三方库** 的综合方案，核心优势如下：

1. 功能层面：满足导航栏基础交互、沉浸式体验、动态主题切换需求；
2. 代码层面：组件化设计提升复用性，清晰 API 降低使用成本；
3. 体验层面：视觉风格统一，适配各类设备，用户体验良好。

该方案既满足项目功能需求，又保证了代码的可维护性和扩展性，是一套完整的导航栏解决方案。

（注：文档部分内容可能由 AI 生成）